

```
import pickle
import numpy as np
import tensorflow
import keras
```

```
# Loading the data from pickle files:
with open("/content/images.p", 'rb') as f:
    X = pickle.load(f)

with open("/content/labels.p", 'rb') as f:
    y = pickle.load(f)
```

```
print("Images:", X.shape)
print("Labels:", y.shape)
```

```
Images: (16082, 48, 48)
Labels: (16082,)
```

```
# Label encoding output.
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

y = le.fit_transform(y)
```

```
y
```

```
array([0, 0, 0, ..., 4, 4, 4])
```

```
# Standardizing the images for CNN Model:
X = X/255
X = X.reshape(-1,48,48,1)
```

```
# Using one-hot encoding to output col:
from keras.utils import to_categorical
y = to_categorical(y)
```

```
# Splitting training and testing datas:
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest = train_test_split(X,y,test_size=0.1)
```

```
print("Xtrain:", xtrain.shape)
print("Ytrain:", ytrain.shape)
print("Xtest:", xtest.shape)
print("Ytest:", ytest.shape)
```

```
Xtrain: (14473, 48, 48, 1)
Ytrain: (14473, 5)
Xtest: (1609, 48, 48, 1)
Ytest: (1609, 5)
```

```
# CNN Model Training:
from keras.models import Sequential
from keras.optimizers import Adam
from keras.layers import Dense, Conv2D, MaxPooling2D, Flatten
```

```
model = Sequential()
# 1st Convolutional Layer:
model.add(Conv2D(45, (3,3), activation='relu',
                 input_shape= (48,48,1)))
model.add(MaxPooling2D(pool_size=(2,2)))

# 2nd Convolutional Layer:
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

# 3rd Convolutional Layer:
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

# Adding flatten layer: converts 2D to 1D
model.add(Flatten())

# Hidden Layer:
model.add(Dense(256, activation='relu'))

# Output Layer:
model.add(Dense(5, activation='softmax'))
model.compile(Adam(learning_rate=0.001),
              loss = 'categorical_crossentropy',
              metrics=['accuracy'])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Wh
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 46, 46, 45)	450
max_pooling2d (MaxPooling2D)	(None, 23, 23, 45)	0
conv2d_1 (Conv2D)	(None, 21, 21, 64)	25,984
max_pooling2d_1 (MaxPooling2D)	(None, 10, 10, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 256)	524,544
dense_1 (Dense)	(None, 5)	1,285

Total params: 626,119 (2.39 MB)

Trainable params: 626,119 (2.39 MB)

Non-trainable params: 0 (0.00 B)

```
history = model.fit(xtrain,ytrain,epochs=20,
                    validation_data=(xtest,ytest))
```

```
Epoch 1/20
453/453 ————— 10s 12ms/step - accuracy: 0.2994 - loss: 1.5451 - val_accuracy: 0.4661 - val_loss: 1.3154
Epoch 2/20
453/453 ————— 3s 6ms/step - accuracy: 0.4807 - loss: 1.2571 - val_accuracy: 0.5090 - val_loss: 1.2146
Epoch 3/20
453/453 ————— 2s 5ms/step - accuracy: 0.5408 - loss: 1.1373 - val_accuracy: 0.5612 - val_loss: 1.1037
Epoch 4/20
453/453 ————— 2s 5ms/step - accuracy: 0.5980 - loss: 1.0148 - val_accuracy: 0.5625 - val_loss: 1.0816
Epoch 5/20
453/453 ————— 2s 5ms/step - accuracy: 0.6315 - loss: 0.9368 - val_accuracy: 0.5985 - val_loss: 1.0421
Epoch 6/20
453/453 ————— 2s 5ms/step - accuracy: 0.6698 - loss: 0.8502 - val_accuracy: 0.6004 - val_loss: 1.0158
Epoch 7/20
453/453 ————— 3s 6ms/step - accuracy: 0.7029 - loss: 0.7688 - val_accuracy: 0.6060 - val_loss: 1.0458
Epoch 8/20
453/453 ————— 2s 5ms/step - accuracy: 0.7450 - loss: 0.6724 - val_accuracy: 0.5960 - val_loss: 1.1101
Epoch 9/20
453/453 ————— 2s 5ms/step - accuracy: 0.7815 - loss: 0.5883 - val_accuracy: 0.6041 - val_loss: 1.1357
Epoch 10/20
453/453 ————— 3s 6ms/step - accuracy: 0.8230 - loss: 0.4891 - val_accuracy: 0.6147 - val_loss: 1.1686
Epoch 11/20
453/453 ————— 5s 6ms/step - accuracy: 0.8526 - loss: 0.3997 - val_accuracy: 0.5948 - val_loss: 1.3396
Epoch 12/20
453/453 ————— 2s 5ms/step - accuracy: 0.8868 - loss: 0.3237 - val_accuracy: 0.6010 - val_loss: 1.4541
Epoch 13/20
453/453 ————— 2s 5ms/step - accuracy: 0.9171 - loss: 0.2440 - val_accuracy: 0.5998 - val_loss: 1.6082
Epoch 14/20
453/453 ————— 2s 5ms/step - accuracy: 0.9373 - loss: 0.1860 - val_accuracy: 0.5973 - val_loss: 1.8828
Epoch 15/20
453/453 ————— 3s 7ms/step - accuracy: 0.9474 - loss: 0.1563 - val_accuracy: 0.6165 - val_loss: 2.0521
Epoch 16/20
```

```
453/453 — 3s 6ms/step - accuracy: 0.9589 - loss: 0.1290 - val_accuracy: 0.6078 - val_loss: 2.1997
Epoch 17/20
453/453 — 2s 5ms/step - accuracy: 0.9658 - loss: 0.1037 - val_accuracy: 0.5848 - val_loss: 2.4050
Epoch 18/20
453/453 — 2s 5ms/step - accuracy: 0.9776 - loss: 0.0756 - val_accuracy: 0.6047 - val_loss: 2.5534
Epoch 19/20
453/453 — 2s 5ms/step - accuracy: 0.9775 - loss: 0.0701 - val_accuracy: 0.5935 - val_loss: 2.7540
Epoch 20/20
453/453 — 3s 6ms/step - accuracy: 0.9705 - loss: 0.0849 - val_accuracy: 0.5693 - val_loss: 2.7142
```

```
# to save the model for deployment:
# .h5 -> HDF5 File Format
# It stores model's optimizer's state, model architecture, training configurations, weights
model.save('emotion_detection_model.h5')
print("Model Saved")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the SavedModel format instead.
Model Saved
```

-